

COLLEGE OF ENGINEERING

THIRUVANANTHAPURAM



COMPUTING LAB 1(221LCS100)
LABORATORY RECORD

DEPARTMENT OF COMPUTER SCIENCE
AND
ENGINEERING

YEAR 2025-26

COLLEGE OF ENGINEERING
THIRUVANANTHAPURAM
COMPUTING LAB 1
(221LCS100)

University Reg.No:
Name:
RollNo:Batch.....
Class: from Page No to Page No

CERTIFIED BONAFIDE RECORD OF WORK DONE BY

.....

External Examiner

Staff Member in Charge

Thiruvananthapuram

Date:.....

EXPERIMENTS

Exp No.	Experiment Name	Date	Sign
1	ID3 ALGORITHM	31-07-2025	
2	NAIVE BAYES CLASSIFIER	07-08-2025	
3	NAIVE BAYES DOCUMENT CLASSIFIER	14-08-2025	
4	HEART DISEASE DIAGNOSIS USING BAYESIAN NETWORK	21-08-2025	
5	CLUSTERING USING EM AND KMEANS ALGORITHM	11-09-2025	
6	K-NEAREST NEIGHBORS ALGORITHM	18-09-2025	
7	5-FOLD CROSS VALIDATION	25-09-2025	
8	SOFTMAX CLASSIFIER (i) kNN (ii) 3 LAYER NEURAL NETWORK	09-10-2025	
9	ANN USING BACKPROPOGATION	09-10-2025	
10	LOGISTIC REGRESSION	21-11-2025	
11	FAMILIARIZATION OF GOOGLE COLAB	21-11-2025	

ID3 ALGORITHM

Aim

To write a program to demonstrate the working of the decision tree based id3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Description

A decision tree is a hierarchical structure that resembles a flowchart, consisting of nodes and branches. Each internal node represents a decision based on a feature, while each leaf node represents a class label or a numerical value in case of regression. The path from the root node to a leaf node represents the decision-making process.

The Iterative Dichotomiser 3 (ID3) algorithm is a decision tree induction algorithm that constructs decision trees using a top-down approach. The main goal of ID3 is to find the most informative attributes at each step to create partitions that yield the highest information gain or decrease in impurity.

Algorithm

1. Take the Entire dataset as an input.
2. Calculate the Entropy of the target variable, As well as the predictor attributes.

$$H(S) = -\sum (P_i \cdot \log_2(P_i))$$

where, P_i represents the fraction of the sample within a particular node.

S – The current dataset.

i – Set of classes in S

The entropy value ranges from 0 to 1. A value of 0 indicates that the dataset is perfectly pure (all instances belong to the same class), while a value of 1 indicates maximum impurity (instances are evenly distributed across different classes).

3. Select the best attribute
 - a. Calculate the information gain for each attribute in the dataset.

$$IG(A, D) = H(S) - \sum_v |S_v|/|S| \times H(S_v)$$

where, $|S|$ is the total number of instances in dataset.

$|S_v|$ is the number of instances in dataset for which attribute D has values v .

$H(S)$ is the entropy of dataset

- b. The attribute with the highest information gain becomes the root node of the decision tree.
4. Repeat the same procedure on every branch until the terminal node (leaf node) of each branch is finalized like shown in the above picture.

Program

```
import pandas as pd
import math
import numpy as np
data = pd.read_csv("weather.csv")
features = [feat for feat in data]
features.remove("play")
features.remove("id")
with open("weather.csv", "w") as f:
    f.write("id,outlook,temperature,humidity,windy,play\n")
    f.write("1,sunny,hot,high,False,no\n")
    f.write("2,sunny,hot,high,True,no\n")
    f.write("3,overcast,hot,high,False,yes\n")
    f.write("4,rainy,mild,high,False,yes\n")
    f.write("5,rainy,cool,normal,False,yes\n")
    f.write("6,rainy,cool,normal,True,no\n")
    f.write("7,overcast,cool,normal,True,yes\n")
    f.write("8,sunny,mild,high,False,no\n")
    f.write("9,sunny,cool,normal,False,yes\n")
    f.write("10,rainy,mild,normal,False,yes\n")
    f.write("11,sunny,mild,normal,True,yes\n")
    f.write("12,overcast,mild,high,True,yes\n")
    f.write("13,overcast,hot,normal,False,yes\n")
    f.write("14,rainy,mild,high,True,no\n")
    def __init__(self):
        self.children = []
        self.value = ""
        self.isLeaf = False
        self.pred = ""
    def entropy(examples):
```

```

pos = 0.0
neg = 0.0
for _, row in examples.iterrows():
    if row["play"] == "yes":
        pos += 1
    else:
        neg += 1
if pos == 0.0 or neg == 0.0:
    return 0.0
else:
    p = pos / (pos + neg)
    n = neg / (pos + neg)
    return -(p * math.log(p, 2) + n * math.log(n, 2))

def info_gain(examples, attr):
    uniq = np.unique(examples[attr])
    #print ("\n",uniq)
    gain = entropy(examples)
    #print ("\n",gain)
    for u in uniq:
        subdata = examples[examples[attr] == u]
        #print ("\n",subdata)
        sub_e = entropy(subdata)
        gain -= (float(len(subdata)) / float(len(examples))) * sub_e
    #print ("\n",gain)
    return gain

def ID3(examples, attrs):
    root = Node()
    max_gain = 0
    max_feat = ""
    for feature in attrs:

```

```

#print ("\n",examples)

gain = info_gain(examples, feature)

if gain > max_gain:
    max_gain = gain
    max_feat = feature

root.value = max_feat

#print ("\nMax feature attr",max_feat)

uniq = np.unique(examples[max_feat])

#print ("\n",uniq)

for u in uniq:
    #print ("\n",u)

    subdata = examples[examples[max_feat] == u]

    #print ("\n",subdata)

    if entropy(subdata) == 0.0:
        newNode = Node()
        newNode.isLeaf = True
        newNode.value = u
        newNode.pred = np.unique(subdata["play"]) # Corrected to "play"
        root.children.append(newNode)
    else:
        dummyNode = Node()
        dummyNode.value = u
        new_attrs = attrs.copy()
        new_attrs.remove(max_feat)
        child = ID3(subdata, new_attrs)
        dummyNode.children.append(child)
        root.children.append(dummyNode)

return root

def printTree(root: Node, depth=0):
    for i in range(depth):

```

```

        print("\t", end="")
    print(root.value, end="")
    if root.isLeaf:
        print(" -> ", root.pred)
    print()
    for child in root.children:
        printTree(child, depth + 1)
def classify(root: Node, new):
    for child in root.children:
        if child.value == new[root.value]:
            if child.isLeaf:
                print ("Predicted Label for new example", new, " is:", child.pred)
                # exit # This line would exit the program, which might not be desired
            else:
                classify (child.children[0], new)
root = ID3(data, features)
print("Decision Tree is:")
printTree(root)
print ("-----")
new = {"outlook":"sunny", "temperature":"hot", "humidity":"normal", "windy":"strong"} #
Changed 'wind' to 'windy'
classify (root, new)

```

Dataset (weather.csv)

A	B	C	D	E	F	
	id	outlook	temperature	humidity	windy	play
	1	sunny	hot	high	FALSE	no
	2	sunny	hot	high	TRUE	no
	3	overcast	hot	high	FALSE	yes
	4	rainy	mild	high	FALSE	yes
	5	rainy	cool	normal	FALSE	yes
	6	rainy	cool	normal	TRUE	no
	7	overcast	cool	normal	TRUE	yes
	8	sunny	mild	high	FALSE	no
	9	sunny	cool	normal	FALSE	yes
	10	rainy	mild	normal	FALSE	yes
	11	sunny	mild	normal	TRUE	yes
	12	overcast	mild	high	TRUE	yes
	13	overcast	hot	normal	FALSE	yes
	14	rainy	mild	high	TRUE	no

Output

Decision Tree is:

outlook

overcast -> ['yes']

rainy

windy

False -> ['yes']

True -> ['no']

sunny

humidity

high -> ['no']

normal -> ['yes']

Predicted Label for new example {'outlook': 'sunny', 'temperature': 'hot', 'humidity': 'normal', 'windy': 'strong'} is: ['yes']

Result

Successfully demonstrated the working of the decision tree based id3 algorithm.

NAIVE BAYES CLASSIFIER

Aim

To write a program to implement the naive bayesian classifier for a sample training data set stored as a .csv file. Compute the accuracy of the classifier, considering few data sets.

Description

Naive Bayes algorithm is a supervised learning algorithm, which is based on Bayes theorem and used for solving classification problems. It is mainly used in text classification that includes a high-dimensional training dataset. It is a probabilistic classifier, which means it predicts on the basis of the probability of an object. Some popular examples of Naive Bayes Algorithm are spam filtration, Sentimental analysis, and classifying articles.

The Naive Bayes algorithm is comprised of two words Naive and Bayes, which can be described as:

- Naive: It is called Naive because it assumes that the occurrence of a certain feature is independent of the occurrence of other features.
- Bayes: It is called Bayes because it depends on the principle of Bayes Theorem.

Bayes' theorem is also known as Bayes' Rule or Bayes' law, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability. The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

$P(A|B)$ is Posterior

probability: Probability of hypothesis

A on the observed event B.

$P(B|A)$ is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true. $P(A)$ is Prior Probability: Probability of hypothesis before observing the evidence. $P(B)$ is Marginal Probability: Probability of Evidence.

Gaussian Naive Bayes

Gaussian Naive Bayes is a type of classification algorithm working on continuous normally distributed features that is based on the Naive Bayes algorithm. For a feature x_i , the likelihood $P(x_i | c)$ is modelled as a Gaussian distribution

$$P(x_i | c) = \frac{1}{\sqrt{2\pi \cdot \text{var}}} \cdot \exp\left(-\frac{(x_i - \text{mean})^2}{2 \cdot \text{var}}\right)$$

Where, mean: The mean value of feature x_i for class c.

var: The variance of feature x_i for class c.

Algorithm

1. Load the Dataset and separate the features (X) and target variable (y).
2. Divide the dataset into training and testing sets.
3. Calculate prior probability for given class labels.
4. Calculate conditional probability with each attribute for each class using Gaussian distribution.
5. Multiply same class conditional probability.
6. Multiply prior probability with step 5 probability.
7. See which class has higher probability, higher probability class belongs to given input set step.
8. Calculate the accuracy of the predictions by comparing them to the true labels from the test set.
9. Output Result

Program

```
import pandas as pd
import numpy as np
import math
from collections import defaultdict

data = pd.read_csv("naiv (2).csv")
print("First 5 rows of the dataset:")
print(data.head())

X = data.iloc[:, :-1].values # all columns except last
y = data.iloc[:, -1].values # last column

np.random.seed(42)
indices = np.arange(len(X))
np.random.shuffle(indices)

split = int(0.8 * len(X))
train_idx, test_idx = indices[:split], indices[split:]

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

class GaussianNB_FromScratch:
    def fit(self, X, y):
```

```

self.classes = np.unique(y)

self.mean = {}
self.var = {}
self.priors = {}

for c in self.classes:
    X_c = X[y == c]
    self.mean[c] = np.mean(X_c, axis=0)
    self.var[c] = np.var(X_c, axis=0) + 1e-6 # avoid zero division
    self.priors[c] = X_c.shape[0] / X.shape[0]

def gaussian_pdf(self, class_idx, x):
    mean = self.mean[class_idx]
    var = self.var[class_idx]
    numerator = np.exp(-((x - mean) ** 2) / (2 * var))
    denominator = np.sqrt(2 * np.pi * var)
    return numerator / denominator

def predict_one(self, x):
    posteriors = {}
    for c in self.classes:
        prior = np.log(self.priors[c])
        conditional = np.sum(np.log(self.gaussian_pdf(c, x)))
        posteriors[c] = prior + conditional
    return max(posteriors, key=posteriors.get)

def predict(self, X):
    return np.array([self.predict_one(x) for x in X])

model = GaussianNB_FromScratch()
model.fit(X_train, y_train)

```

```

y_pred = model.predict(X_test)
accuracy = np.mean(y_pred == y_test)
print(f'\nAccuracy of Naive Bayes classifier: {accuracy * 100:.2f} %')
comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print("\nSample Predictions:")
print(comparison.head())

```

Dataset (naiv.csv)

1 to 10 of 995 entries 

glucose	bloodpressure	diabetes
40	85	0
40	92	0
45	63	1
45	80	0
40	73	1
45	82	0
40	85	0
30	63	1
65	65	1
45	82	0

Output

First 5 rows of the dataset:

	glucose	bloodpressure	diabetes
0	40	85	0
1	40	92	0
2	45	63	1
3	45	80	0
4	40	73	1

Accuracy of Naive Bayes classifier: 94.47%

Sample Predictions:

	Actual	Predicted
0	1	1
1	0	0
2	0	1
3	1	1
4	1	1

Result

Successfully implemented the naive bayes classifier for a sample training data set and computed the accuracy of the classifier.

NAIVE BAYES DOCUMENT CLASSIFIER

Aim

To write a program to classify a set of documents using the naive bayesian classifier model and to calculate the accuracy, precision, and recall for the data set.

Description

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem, which describes the relationship between conditional probabilities. It is particularly well-suited for text classification problems because of its simplicity, efficiency, and strong performance on high-dimensional data, such as word vectors.

Bayes' Theorem: The foundation of Naive Bayes is Bayes' Theorem:

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Where, $P(c|x)$: The probability of class c given the feature vector x .

$P(x|c)$: The probability of the feature vector x given the class c (likelihood).

$P(c)$: The prior probability of class c (how likely the class is overall).

$P(x)$: The probability of the feature vector x .

Naive Assumption: The algorithm assumes that all features are conditionally independent given the class. This simplifies the computation of the likelihood $P(x|c)$.

Gaussian Naive Bayes

Gaussian Naive Bayes is a type of classification algorithm working on continuous normally distributed features that is based on the Naive Bayes algorithm. For a feature x_i , the likelihood $P(x_i|c)$ is modelled as a Gaussian distribution

$$P(x_i|c) = \frac{1}{\sqrt{2\pi \cdot \text{var}}} \cdot \exp\left(-\frac{(x_i - \text{mean})^2}{2 \cdot \text{var}}\right)$$

Where, mean: The mean value of feature x_i for class c .

var: The variance of feature x_i for class c .

To avoid computational underflow when multiplying many small probabilities, the algorithm computes the log probabilities:

$$\log P(x|c) = -0.5 \cdot \sum \log(2\pi \cdot \text{var}) - 0.5 \cdot \sum \frac{(x - \text{mean})^2}{\text{var}}$$

Performance Evaluation Metrics

1. Accuracy: Measures overall correctness

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

2. Precision: Measures the proportion of true positives out of all predicted positives

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

3. Recall: Measures the proportion of true positives out of all actual positives

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Algorithm

1. Load the Dataset and Separate the Features (X) and Target Variable (y).
2. Clean the Text Data.
3. Divide the Dataset into Training and Testing Sets
4. Convert the text data into numerical features
5. Calculate the mean, variance, and prior probability for each class based on the training data.
6. For each test sample, calculate log probabilities for each class using Gaussian probability and prior probabilities, and assign the class with the highest log probability.
7. Evaluate the model using accuracy, precision, and recall metrics.
8. Output the result

Program

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix
from sklearn.feature_extraction.text import CountVectorizer

df= pd.read_csv('spam.csv',encoding='latin-1')
df= df[['v1', 'v2']]

# Rename columns for clarity
df.columns = ['label', 'message']

# Convert labels to numerical (0 for ham, 1 for spam)
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

print("\nSample Data Set\n",df.head())
print("\ncolumns\n",df.columns)
# Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(df['message'], df['label'], test_size=0.8,
random_state=42)
```

```
# Vectorize the text data
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

```
# Train a Naive Bayes classifier
model = MultinomialNB()
model.fit(X_train_vec, y_train)
```

```
# Make predictions
y_pred = model.predict(X_test_vec)
```

```
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
print("\nAccuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("Confusion Matrix:\n", conf_matrix)
```

Dataset (spam.csv)

The SMS Spam Collection is a set of SMS tagged messages that have been collected for SMS Spam research. It contains one set of SMS messages in English of 5,574 messages, tagged according being ham (legitimate) or spam.

1	v1	
2	ham	Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...
3	ham	Ok lar... Joking wif u oni...
4	spam	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 0845281007Sover18's
5	ham	U dun say so early hor... U c already then say...
6	ham	Nah I don't think he goes to usf, he lives around here though
7	spam	FreeMsg Hey there darling it's been 3 week's now and no word back! I'd like some fun you up for it still? Tb ok! XxX std chgs to send, â€1.50 to rcv
8	ham	Even my brother is not like to speak with me. They treat me like aids patient.
9	ham	As per your request 'Melle Melle (Oru Minnaminunginte Nurgu Vettam)' has been set as your callertune for all Callers. Press *9 to copy your friends Callertune
10	spam	WINNER!! As a valued network customer you have been selected to receive a £900 prize reward! To claim call 09061701461. Claim code KL341. Valid 12 hours only.
11	spam	Had your mobile 11 months or more? U R entitled to Update to the latest colour mobiles with camera for Free! Call The Mobile Update Co FREE on 08002986030
12	ham	I'm gonna be home soon and i don't want to talk about this stuff anymore tonight, k? I've cried enough today.
13	spam	SIX chances to win CASH! From 100 to 20,000 pounds txt> CSH11 and send to 87575. Cost 150p/day, 6days, 16+ TsandCs apply Reply HL 4 info
14	spam	URGENT! You have won a 1 week FREE membership in our â€100,000 Prize Jackpot! Txt the word: CLAIM to No: 81010 Tâ€c www.dbuk.net LCCLTD POBOX 4403LDNW1A7RW18
15	ham	I've been searching for the right words to thank you for this breather. I promise i wont take your help for granted and will fulfil my promise. You have been wonderful and a blessing at all times.

Output

```
Sample Data Set
  label message
0      0  Go until jurong point, crazy.. Available only ...
1      0               Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...

columns
Index(['label', 'message'], dtype='object')

Accuracy: 0.9782413638402871
Precision: 0.9659735349716446
Recall: 0.8661016949152542
Confusion Matrix:
[[3850  18]
 [ 79 511]]
```

Result

Successfully classified of a set of documents using the Naive Bayes classifier model. The performance of the classifier was evaluated by calculating the accuracy, precision, and recall.

HEART DISEASE DIAGNOSIS USING BAYESIAN NETWORK

Aim

To write a program to construct a bayesian network considering medical data. Use the model to demonstrate the diagnosis of heart patients using standard heart disease data set (you can use python ml library classes/api).

Description

Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem, which describes the relationship between conditional probabilities. It is particularly well-suited for text classification problems because of its simplicity, efficiency, and strong performance on high-dimensional data, such as word vectors.

Bayes' Theorem: The foundation of Naive Bayes is Bayes' Theorem:

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Where, $P(c|x)$: The probability of class c given the feature vector x .

$P(x|c)$: The probability of the feature vector x given the class c (likelihood).

$P(c)$: The prior probability of class c (how likely the class is overall).

$P(x)$: The probability of the feature vector x .

Naive Assumption: The algorithm assumes that all features are conditionally independent given the class. This simplifies the computation of the likelihood $P(x|c)$.

Gaussian Naive Bayes

Gaussian Naive Bayes is a type of classification algorithm working on continuous normally distributed features that is based on the Naive Bayes algorithm. For a feature x_i , the likelihood $P(x_i|c)$ is modelled as a Gaussian distribution

$$P(x_i|c) = \frac{1}{\sqrt{2\pi \cdot \text{var}}} \cdot \exp\left(-\frac{(x_i - \text{mean})^2}{2 \cdot \text{var}}\right)$$

Where, mean: The mean value of feature x_i for class c .

var: The variance of feature x_i for class c .

Algorithm

1. Load the Dataset and separate the features (X) and target variable (y).
2. Divide the dataset into training and testing sets.
3. Calculate prior probability for given class labels.
4. Calculate conditional probability with each attribute for each class using Gaussian distribution.
5. Multiply same class conditional probability.
6. Multiply prior probability with step 5 probability.
7. See which class has higher probability, higher probability class belongs to given input set step.
8. Calculate the accuracy of the predictions by comparing them to the true labels from the test set.
9. Output Result

Program

```
!pip install pgmpy pandas scikit-learn numpy networkx matplotlib --quiet
import pandas as pd
import numpy as np
from pgmpy.models import DiscreteBayesianNetwork # Corrected import
from pgmpy.estimators import HillClimbSearch, BayesianEstimator
from pgmpy.inference import VariableElimination
from sklearn.preprocessing import LabelEncoder
import networkx as nx
import matplotlib.pyplot as plt
df = pd.read_csv("heart.csv")
print(" Dataset loaded successfully!\n")
print(df.head())
label_encoders = {}
for column in df.columns:
    if df[column].dtype == 'object':
        le = LabelEncoder()
        df[column] = le.fit_transform(df[column].astype(str))
        label_encoders[column] = le

df = df.fillna(df.median(numeric_only=True))
print("\n Data preprocessing completed. Shape:", df.shape)

print("\n Learning Bayesian Network structure...")
hc = HillClimbSearch(df)
best_model = hc.estimate()
print(" Structure learned successfully!")
print("Edges in the learned network:", best_model.edges())
model = DiscreteBayesianNetwork(best_model.edges()) # Corrected
model.fit(df, estimator=BayesianEstimator, prior_type='BDeu')
print("\n Bayesian Network trained successfully!")
plt.figure(figsize=(10,6))
```

```

G = nx.DiGraph()
G.add_edges_from(model.edges())
pos = nx.spring_layout(G, seed=42) # Layout for visualization
nx.draw(G, pos, with_labels=True, node_size=3000, node_color='skyblue', font_size=12,
font_weight='bold', arrowsize=20)
plt.title("Bayesian Network Structure for Heart Disease")
plt.show()
inference = VariableElimination(model)
target_col = None
for col in df.columns:
    if col.lower() in ['target', 'disease', 'output']:
        target_col = col
        break
if target_col:
    print(f"\n Performing heart disease diagnosis for column: '{target_col}'")
    query_example = df.sample(1).iloc[0].to_dict()
    print("\n Example patient data:")
    print(query_example)
    valid_evidence = {k: v for k, v in query_example.items() if k in model.nodes() and k != target_col}
    result = inference.map_query([target_col], evidence=valid_evidence)
    print(f"\n Predicted Heart Disease Diagnosis → {target_col} = {result[target_col]}")
else:
    print("\n No column named 'target', 'disease', or 'output' found. Please rename your target column accordingly.")

print("\n Bayesian Network Heart Diagnosis completed successfully!")

```

Dataset(heartdata.csv)

age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
60	1	3	145	233	1	0	150	0	2.3	0	0	1	1
35	1	2	130	250	0	1	187	0	3.5	0	0	2	1
41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
55	1	1	120	236	0	1	178	0	0.8	2	0	2	1
56	0	0	120	354	0	1	163	1	0.6	2	0	2	1
55	1	0	140	192	0	1	148	0	0.4	1	0	1	1
56	0	1	140	294	0	0	153	0	1.3	1	0	2	1
44	1	1	120	263	0	1	173	0	0	2	0	3	1
52	1	2	172	199	1	1	162	0	0.5	2	0	3	1
57	1	2	150	168	0	1	174	0	1.6	2	0	2	1
54	1	0	140	239	0	1	160	0	1.2	2	0	2	1
48	0	2	130	275	0	1	139	0	0.2	2	0	2	1
49	1	1	130	266	0	1	171	0	0.6	2	0	2	1
64	1	3	110	211	0	0	144	1	1.8	1	0	2	1
55	0	3	150	283	1	0	162	0	1	2	0	2	1
50	0	2	120	219	0	1	158	0	1.6	1	0	2	1
58	0	2	120	340	0	1	172	0	0	2	0	2	1

Output

```
pomegranate not present - attempting to install it (this may take a minute)...
Could not install pomegranate. Falling back to Naive Bayes (sklearn).
Loaded dataset from: /content/heart.csv
Dataset shape: (289, 14)
Columns: ['age', 'sex', 'cp', 'trtbps', 'chol', 'fbs', 'restecg', 'thalachh', 'exng', 'oldpeak', 'slp', 'caa', 'thall', 'output']
Continuous features to discretize: ['age', 'trtbps', 'chol', 'thalachh', 'oldpeak']
Value counts for discretized features (example):
age {0: 95, 1: 97, 2: 97}
trtbps {0: 94, 1: 92, 2: 103}
chol {0: 94, 1: 98, 2: 97}
thalachh {0: 92, 1: 93, 2: 104}
oldpeak {0: 96, 1: 96, 2: 97}
Train/Test shapes: (202, 14) (87, 14)
Target distribution (train):
  output
1    0.569307
0    0.430693
Name: proportion, dtype: float64
Target distribution (test):
  output
1    0.574713
0    0.425287
Name: proportion, dtype: float64
```

Using Naive Bayes (sklearn CategoricalNB) as fallback demonstration.

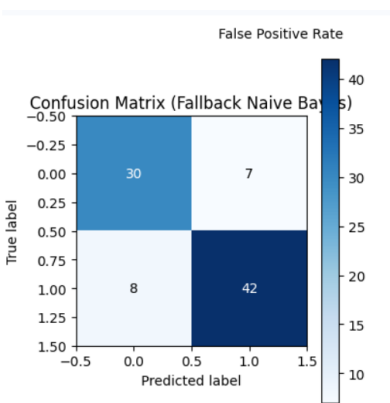
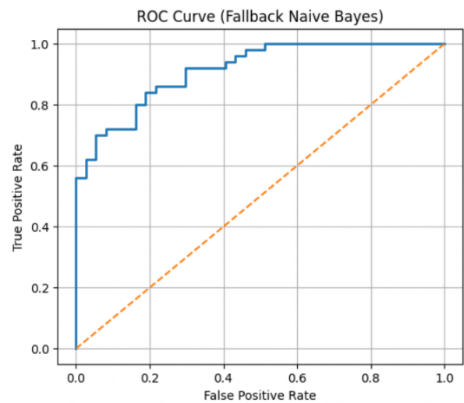
```
=== Naive Bayes (fallback) Evaluation on Test Set ===
Accuracy: 0.8276
ROC AUC: 0.9135
```

Classification report:

	precision	recall	f1-score	support
0	0.7895	0.8108	0.8000	37
1	0.8571	0.8400	0.8485	50
accuracy			0.8276	87
macro avg	0.8233	0.8254	0.8242	87
weighted avg	0.8284	0.8276	0.8279	87

```
Confusion matrix:
[[30  7]
 [ 8 42]]
```

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output	pred_prob_output1	pred_output
41	0	1	1	1	1	0	0	2	0	1	1	0	2	1	0.979499	1
28	2	0	2	2	2	1	0	1	0	1	2	1	2	1	0.937338	1
270	0	1	0	0	1	0	0	1	0	1	2	0	3	0	0.712758	1
152	2	1	3	2	1	0	0	1	0	1	1	0	3	1	0.302949	0
70	1	1	2	0	1	0	0	1	0	1	1	0	3	1	0.670956	1
62	1	1	3	0	0	0	0	2	0	0	1	0	1	1	0.909440	1
262	1	1	0	1	2	0	1	0	1	2	1	2	3	0	0.000237	0
194	2	1	2	2	0	0	0	1	0	2	1	0	2	0	0.684071	1
34	1	1	3	1	0	0	0	0	1	2	2	1	2	1	0.161127	0
85	2	0	2	0	2	0	0	1	0	2	1	0	3	1	0.321907	0
253	2	1	0	0	2	0	0	0	1	1	1	2	2	0	0.003218	0
107	0	0	0	2	1	0	0	1	1	1	1	0	2	1	0.520007	1



RESULT

Successfully implemented the Bayesian Network for a standard heart disease dataset and computed the accuracy of the model in diagnosing heart patients.

CLUSTERING USING EM AND K-MEANS ALGORITHMS

Aim

To apply the EM algorithm and the k-Means algorithm for clustering a dataset stored in a .csv file, using Python machine learning libraries. The goal is to compare the results of both algorithms in terms of clustering quality and performance.

Description

The task of grouping data points based on their similarity with each other is called Clustering or Cluster Analysis. This method is defined under the branch of Unsupervised Learning, which aims at gaining insights from unlabelled data points.

Expectation-Maximization algorithm

The Expectation-Maximization (EM) algorithm is an iterative optimization method that combines different unsupervised machine learning algorithms to find maximum likelihood or maximum posterior estimates of parameters in statistical models that involve unobserved latent variables. The EM algorithm is commonly used for latent variable models and can handle missing data. It consists of an estimation/expectation step (E-step) and a maximization step (M-step), forming an iterative process to improve model fit.

- Expectation step (E - step): It involves the estimation of all missing values in the dataset so that after completing this step, there should not be any missing value.
- Maximization step (M - step): This step involves the use of estimated data in the E-step and updating the parameters.
- Repeat E-step and M-step until the convergence of the values occurs.

K-Means clustering algorithm

K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters. Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on. It is an iterative algorithm that divides the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties. It is a centroid-based algorithm, where each cluster is associated with a centroid. The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.

The algorithm takes the unlabeled dataset as input, divides the dataset into k-number of clusters, and repeats the process until it does not find the best clusters. The value of k should be predetermined in this algorithm. The k-means clustering algorithm mainly performs two tasks:

Determines the best value for K center points or centroids by an iterative process.

- Assigns each data point to its closest k-center. Those data points which are near to the particular k-center, create a cluster.

Hence each cluster has datapoints with some commonalities, and it is away from other clusters.

Algorithm

1. Import essential libraries for data manipulation, clustering, and visualization.
2. Load the dataset (in CSV format) into a pandas DataFrame.
3. Select only the numeric features for clustering, ignoring any non-numeric columns.
4. Scale the features to ensure that all features contribute equally to the clustering algorithm.
5. Apply the K-Means clustering algorithm, to divide the data into clusters.
6. Apply the Gaussian Mixture Model (EM algorithm) to model the data using components (clusters).
7. Compute the Silhouette Score to evaluate how well-defined the clusters are for both K-Means and the EM algorithm. A higher Silhouette Score indicates better clustering quality.
8. Visualize the clustering results by plotting the data points in 2D and colouring them according to the assigned cluster labels for both algorithms.

Program

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
df = pd.read_csv('iris.csv')
print("First 5 rows of dataset:\n", df.head())

X = df.drop('variety', axis=1).values
else:
    X = df.values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Corrected n_init value to suppress warning
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X_scaled)
kmeans_silhouette = silhouette_score(X_scaled, kmeans_labels)
print(f"k-Means Silhouette Score: {kmeans_silhouette:.3f}")
gmm = GaussianMixture(n_components=3, random_state=42)
gmm_labels = gmm.fit_predict(X_scaled)
gmm_silhouette = silhouette_score(X_scaled, gmm_labels)
print(f"EM (GMM) Silhouette Score: {gmm_silhouette:.3f}")

plt.figure(figsize=(12,5))
```

```

plt.subplot(1,2,1)
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=kmeans_labels, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1], color='red', marker='X', s=200)
plt.title('k-Means Clustering (Iris)')
plt.xlabel(df.columns[0])
plt.ylabel(df.columns[1])
plt.subplot(1,2,2)
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=gmm_labels, cmap='viridis', s=50)
plt.title('EM Clustering (GMM, Iris)')
plt.xlabel(df.columns[0])
plt.ylabel(df.columns[1])
plt.tight_layout()
plt.show()
print("\nComparison:")
if gmm_silhouette > kmeans_silhouette:
    print("EM (GMM) clustering performs slightly better than k-Means based on silhouette score.")
elif gmm_silhouette < kmeans_silhouette:
    print("k-Means clustering performs slightly better than EM (GMM) based on silhouette score.")
else:
    print("Both clustering algorithms perform similarly based on silhouette score.")

```

Iris Dataset (iris.csv)

sepal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
4.6	3.4	1.4	0.3	Setosa
5	3.4	1.5	0.2	Setosa
4.4	2.9	1.4	0.2	Setosa
4.9	3.1	1.5	0.1	Setosa
5.4	3.7	1.5	0.2	Setosa
4.8	3.4	1.6	0.2	Setosa
4.8	3	1.4	0.1	Setosa
4.3	3	1.1	0.1	Setosa
5.8	4	1.2	0.2	Setosa
5.7	4.4	1.5	0.4	Setosa

Output

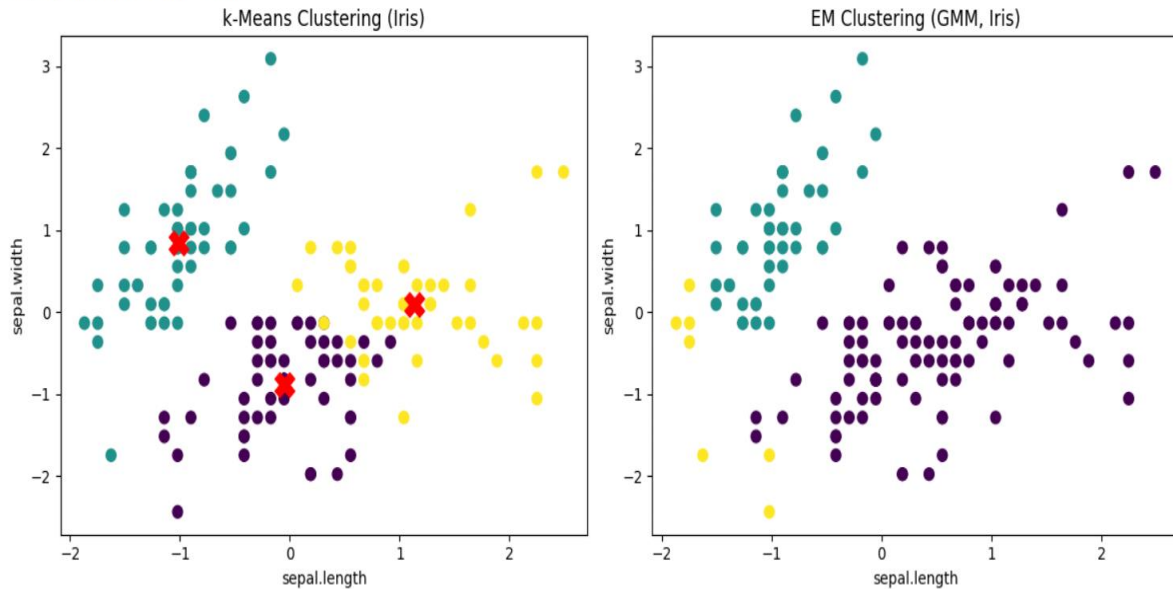
Silhouette Score for K-Means: 0.46

Silhouette Score for EM Algorithm: 0.37

First 5 rows of dataset:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

k-Means Silhouette Score: 0.460
EM (GMM) Silhouette Score: 0.475



Comparison:
EM (GMM) clustering performs slightly better than k-Means based on silhouette score.

Result

The clustering quality of the dataset was evaluated successfully using both the Expectation-Maximization (EM) and k-Means algorithms, with the Silhouette Score used to compare their performance.

K-NEAREST NEIGHBORS ALGORITHM

Aim

To write a program to implement k-nearest neighbour algorithm to classify the iris dataset. Print both correct and wrong predictions (python ml library classes can be used for this problem).

Description

The **K-Nearest Neighbors (KNN) algorithm** is a supervised machine learning method employed to tackle classification and regression problems. It finds intense application in pattern recognition, data mining, and intrusion detection. It is widely disposable in real-life scenarios since it is non-parametric, meaning it does not make any underlying assumptions about the distribution of data. It can also handle both numerical and categorical data, making it a flexible choice for various types of datasets in classification and regression tasks.

The K-NN algorithm works by finding the K nearest neighbors to a given data point based on a distance metric, such as Euclidean distance. The class or value of the data point is then determined by the majority vote or average of the K neighbors.

The distance metric used to measure the similarity between two data points is an essential factor that affects the KNN algorithm's performance. The most commonly used distance metrics are Euclidean distance, Manhattan distance, and Minkowski distance.

KNN algorithm uses 'feature similarity' to predict the values of new datapoints which further means that the new data point will be assigned a value based on how closely it matches the points in the training set. We can understand its working with the help of following steps –

- **Step 1** – The first step of KNN is to load the training as well as test data.
- **Step 2** – Next, choose the value of K i.e. the nearest data points. K can be any integer.
- **Step 3** – For each point in the test data do the following –
 - 3.1** – Calculate the distance between test data and each row of training data with the help of any of the method namely: Euclidean, Manhattan or Hamming distance. The most commonly used method to calculate distance is Euclidean.
 - 3.2** – Now, based on the distance value, sort them in ascending order.
 - 3.3** – Next, it will choose the top K rows from the sorted array.
 - 3.4** – Now, it will assign a class to the test point based on most frequent class of these rows.
- **Step 4** – End

Distance Metrics Used in KNN Algorithm

KNN algorithm helps us identify the nearest points or the groups for a query point. But to determine the closest groups or the nearest points for a query point we need some metric. For this purpose, distance metrics given below can be used :

1. Euclidean Distance

This is the cartesian distance between the two points which are in the plane/hyperplane. Euclidean distance can also be visualized as the length of the straight line that joins the two points which are into consideration. This metric helps us calculate the net displacement done between the two states of an object.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

2. Manhattan Distance

Manhattan Distance metric is generally used when we are interested in the total distance travelled by the object instead of the displacement. This metric is calculated by summing the absolute difference between the coordinates of the points in n-dimensions.

$$distance = \sum_{i=1}^n |p_i - q_i|$$

3. Minkowski Distance

The Euclidean, as well as the Manhattan distance, are special cases of the Minkowski distance.

$$dist = \left(\sum_{k=1}^n |p_k - q_k|^r \right)^{\frac{1}{r}}$$

From the formula above we can say that when $r = 2$ then it is the same as the formula for the Euclidean distance and when $r = 1$ then we obtain the formula for the Manhattan distance.

Iris Dataset

Iris dataset is the dataset used in this K-Nearest Neighbors algorithm. It consists of 150 instances and features four measurements taken from iris flowers: sepal length, sepal width, petal length, and petal width, all measured in centimeters. The dataset includes three distinct species of iris flowers: Iris Setosa, Iris Versicolor, and Iris Virginica, making it a multiclass classification problem.

Downloaded the dataset from: <https://www.kaggle.com/datasets/uciml/iris>

Algorithm

1. Import necessary libraries for data handling, model training, and evaluation.
2. Load the dataset into a DataFrame.
3. Separate the dataset into independent variables (features) and the dependent variable (target).
4. Split the data into training and testing sets using a suitable ratio
5. Set the number of neighbors (k) and initialize the K-Nearest Neighbors (KNN) classifier.
6. Fit the KNN model to the training data.
7. Use the trained KNN model to predict the target values for the test data.
8. Compare predicted values with the actual target values. Also separate correct and incorrect predictions.
9. Output results, such as correct predictions, incorrect predictions, and their details.
10. Compute accuracy to evaluate the model.

Program

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data=pd.read_csv('iris.csv')

X=data.iloc[:, :-1]
y=data.iloc[:, -1]

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.4,random_state=42)

k=3
knn=KNeighborsClassifier(n_neighbors=k)

knn.fit(X_train,y_train)

y_pred = knn.predict(X_test)
correct_predictions=[]
wrong_predictions=[]

for i in range(len(y_test)):
    if y_test.iloc[i]==y_pred[i]:
        correct_predictions.append((X_test.iloc[i],y_test.iloc[i],y_pred[i]))
    else:
```

```

wrong_predictions.append((X_test.iloc[i],y_test.iloc[i],y_pred[i]))

print("Correct Pedictions:")
for data in correct_predictions:
    print(f'Features: {data[0].values} | True Label: {data[1]} | Predicted Label: {data[2]}')

print("\nWrong Predictions:")
for data in wrong_predictions:
    print(f'Features: {data[0].values} | True Label: {data[1]} | Predicted Label: {data[2]}')

accuracy = accuracy_score(y_test,y_pred)
print(f"\nAccuracy: {accuracy*100:.2f} %")

```

Iris Dataset (iris.csv)

sepal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
4.6	3.4	1.4	0.3	Setosa
5	3.4	1.5	0.2	Setosa
4.4	2.9	1.4	0.2	Setosa
4.9	3.1	1.5	0.1	Setosa
5.4	3.7	1.5	0.2	Setosa
4.8	3.4	1.6	0.2	Setosa
4.8	3	1.4	0.1	Setosa
4.3	3	1.1	0.1	Setosa
5.8	4	1.2	0.2	Setosa
5.7	4.4	1.5	0.4	Setosa

Output

```
Correct Pedictions:
Features:[6.1 2.8 4.7 1.2]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.7 3.8 1.7 0.3]| True Label:Setosa| Predicted Label:Setosa
Features:[7.7 2.6 6.9 2.3]| True Label:Virginica| Predicted Label:Virginica
Features:[6. 2.9 4.5 1.5]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.8 2.8 4.8 1.4]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.4 3.4 1.5 0.4]| True Label:Setosa| Predicted Label:Setosa
Features:[5.6 2.9 3.6 1.3]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.9 3.1 5.1 2.3]| True Label:Virginica| Predicted Label:Virginica
Features:[6.2 2.2 4.5 1.5]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.8 2.7 3.9 1.2]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.5 3.2 5.1 2. ]| True Label:Virginica| Predicted Label:Virginica
Features:[4.8 3. 1.4 0.1]| True Label:Setosa| Predicted Label:Setosa
Features:[5.5 3.5 1.3 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[4.9 3.1 1.5 0.1]| True Label:Setosa| Predicted Label:Setosa
Features:[5.1 3.8 1.5 0.3]| True Label:Setosa| Predicted Label:Setosa
Features:[6.3 3.3 4.7 1.6]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.5 3. 5.8 2.2]| True Label:Virginica| Predicted Label:Virginica
Features:[5.6 2.5 3.9 1.1]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.7 2.8 4.5 1.3]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.4 2.8 5.6 2.2]| True Label:Virginica| Predicted Label:Virginica
Features:[4.7 3.2 1.6 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[6.1 3. 4.9 1.8]| True Label:Virginica| Predicted Label:Virginica
Features:[5. 3.4 1.6 0.4]| True Label:Setosa| Predicted Label:Setosa
Features:[6.4 2.8 5.6 2.1]| True Label:Virginica| Predicted Label:Virginica
Features:[7.9 3.8 6.4 2. ]| True Label:Virginica| Predicted Label:Virginica
Features:[6.7 3. 5.2 2.3]| True Label:Virginica| Predicted Label:Virginica
Features:[6.7 2.5 5.8 1.8]| True Label:Virginica| Predicted Label:Virginica
Features:[6.8 3.2 5.9 2.3]| True Label:Virginica| Predicted Label:Virginica
Features:[4.8 3. 1.4 0.3]| True Label:Setosa| Predicted Label:Setosa
Features:[4.8 3. 1.4 0.3]| True Label:Setosa| Predicted Label:Setosa
Features:[4.8 3.1 1.6 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[4.6 3.6 1. 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[5.7 4.4 1.5 0.4]| True Label:Setosa| Predicted Label:Setosa
Features:[6.7 3.1 4.4 1.4]| True Label:Versicolor| Predicted Label:Versicolor
Features:[4.8 3.4 1.6 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[4.4 3.2 1.3 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[6.3 2.5 5. 1.9]| True Label:Virginica| Predicted Label:Virginica
Features:[6.4 3.2 4.5 1.5]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.2 3.5 1.5 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[5. 3.6 1.4 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[5.2 4.1 1.5 0.1]| True Label:Setosa| Predicted Label:Setosa
Features:[5.8 2.7 5.1 1.9]| True Label:Virginica| Predicted Label:Virginica
Features:[6. 3.4 4.5 1.6]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.7 3.1 4.7 1.5]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.4 3.9 1.3 0.4]| True Label:Setosa| Predicted Label:Setosa
Features:[5.4 3.7 1.5 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[5.5 2.4 3.7 1. ]| True Label:Versicolor| Predicted Label:Versicolor
Features:[6.4 3.1 5.5 1.8]| True Label:Virginica| Predicted Label:Virginica
Features:[6.6 3. 4.4 1.4]| True Label:Versicolor| Predicted Label:Versicolor
Features:[7.2 3.6 6.1 2.5]| True Label:Virginica| Predicted Label:Virginica
Features:[5.7 2.9 4.2 1.3]| True Label:Versicolor| Predicted Label:Versicolor
Features:[7.6 3. 6.6 2.1]| True Label:Virginica| Predicted Label:Virginica
Features:[5.6 3. 4.5 1.5]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.1 3.5 1.4 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[7.7 2.8 6.7 2. ]| True Label:Virginica| Predicted Label:Virginica
Features:[5.8 2.7 4.1 1. ]| True Label:Versicolor| Predicted Label:Versicolor
Features:[5.2 3.4 1.4 0.2]| True Label:Setosa| Predicted Label:Setosa
Features:[5. 3.5 1.3 0.3]| True Label:Setosa| Predicted Label:Setosa
```

```
Features:[5.1 3.8 1.9 0.4]| True Label:Setosa| Predicted Label:Setosa  
Features:[5.  2.  3.5 1.  ]| True Label:Versicolor| Predicted Label:Versicolor  
  
Wrong Predictions:  
Features:[6.3 2.8 5.1 1.5]| True Label:Virginica| Predicted Label:Versicolor  
  
Accuracy:98.33%
```

Result

Successfully implemented k-nearest neighbors algorithm to classify the iris dataset and displayed both correct and wrong predictions.

5-FOLD CROSS VALIDATION

Aim

To write a program to implement 5-fold cross validation on a given dataset, compare the accuracy, precision, recall, and f1-score for your data set for different folds.

Description

Cross validation is a technique used in machine learning to evaluate the performance of a model on unseen data. It involves dividing the available data into multiple folds or subsets, using one of these folds as a validation set, and training the model on the remaining folds. This process is repeated multiple times, each time using a different fold as the validation set. Finally, the results from each validation step are averaged to produce a more robust estimate of the model's performance.

K-fold cross-validation technique is basically a method of resampling the data set in order to evaluate a machine learning model. In this technique, the parameter K refers to the number of different subsets that the given data set is to be split into. Further, K-1 subsets are used to train the model and the left out subsets are used as a validation set.

Steps involved in the K-fold Cross Validation :

1. Split the data set into K subsets randomly
2. For each one of the developed subsets of data points
 - Treat that subset as the validation set
 - Use all the rest subsets for training purpose
 - Training of the model and evaluate it on the validation set or test set
 - Calculate prediction error
3. Repeat the above step K times i.e., until the model is not trained and tested on all subsets
4. Generate overall prediction error by taking the average of prediction errors in every case
- 5.

Algorithm

1. Load dataset into a DataFrame.
2. Separate the features (X) and labels (Y).
3. Initialize the classifier model .
4. Train the model using fit() with features and labels.
5. Define the evaluation metrics (accuracy, precision, recall, F1-score).

6. Perform 5-fold cross-validation
7. For each fold, print the evaluation metrics .
8. Calculate and print the average of each metric across all folds.

Program

```
import pandas as pd
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.tree import DecisionTreeClassifier

data = pd.read_csv("iris.csv") # make sure the file name matches yours
print("Dataset loaded successfully!")
print(data.head())
target_col = 'species' if 'species' in data.columns else data.columns[-1]

X = data.drop(target_col, axis=1)
y = data[target_col]
kf = KFold(n_splits=5, shuffle=True, random_state=42)
accuracy_list, precision_list, recall_list, f1_list = [], [], [], []
fold = 1
for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    model = DecisionTreeClassifier(criterion='entropy', random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='macro')
    rec = recall_score(y_test, y_pred, average='macro')
    f1 = f1_score(y_test, y_pred, average='macro')

    accuracy_list.append(acc)
    precision_list.append(prec)
    recall_list.append(rec)
    f1_list.append(f1)

    print(f"\nFold {fold}:")
    print(f"Accuracy : {acc:.3f}")
    print(f"Precision: {prec:.3f}")
    print(f"Recall : {rec:.3f}")
    print(f"F1-score : {f1:.3f}")
    fold += 1

print("\n--- Overall Performance (Average of 5 Folds) ---")
print(f"Mean Accuracy : {sum(accuracy_list)/5:.3f}")
print(f"Mean Precision: {sum(precision_list)/5:.3f}")
print(f"Mean Recall : {sum(recall_list)/5:.3f}")
print(f"Mean F1-score : {sum(f1_list)/5:.3f}")
```

Iris Dataset (iris.csv)

sepal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
4.6	3.4	1.4	0.3	Setosa
5	3.4	1.5	0.2	Setosa
4.4	2.9	1.4	0.2	Setosa
4.9	3.1	1.5	0.1	Setosa
5.4	3.7	1.5	0.2	Setosa
4.8	3.4	1.6	0.2	Setosa
4.8	3	1.4	0.1	Setosa
4.3	3	1.1	0.1	Setosa
5.8	4	1.2	0.2	Setosa
5.7	4.4	1.5	0.4	Setosa

OUTPUT

Dataset loaded successfully!

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

Fold 1:

Accuracy : 1.000

Precision: 1.000

Recall : 1.000

F1-score : 1.000

Fold 2:

Accuracy : 0.967

Precision: 0.970

Recall : 0.952

F1-score : 0.958

Fold 3:

Accuracy : 0.933

Precision: 0.933

Recall : 0.933

F1-score : 0.926

```
Fold 4:  
Accuracy : 0.933  
Precision: 0.939  
Recall   : 0.939  
F1-score : 0.939
```

```
Fold 5:  
Accuracy : 0.933  
Precision: 0.949  
Recall   : 0.944  
F1-score : 0.942
```

```
--- Overall Performance (Average of 5 Folds) ---  
Mean Accuracy : 0.953  
Mean Precision: 0.958  
Mean Recall   : 0.954  
Mean F1-score : 0.953
```

Result

Successfully implemented 5-fold cross validation on a given dataset, also compared the accuracy, precision, recall, and f1 -score for data set for different folds

SOFTMAX CLASSIFIER

Aim

To implement and compare the performance of two machine learning models for image classification on the CIFAR-10 dataset using a softmax classifier:

- (i) K-Nearest Neighbors (KNN) and
- (ii) a 3-layer fully connected Neural Network

Description

Image classification is a fundamental task in computer vision that involves assigning labels to images based on their content. The CIFAR-10 dataset is a standard benchmark dataset containing 60,000 color images grouped into 10 object categories, such as airplanes, automobiles, birds, cats, and ships.

This experiment implements two distinct classification approaches:

1. K-NearestNeighbors(KNN):
A non-parametric, instance-based learning method that classifies test samples based on the majority class among the nearest training samples in feature space. The dataset images are flattened into feature vectors. Due to KNN's computational cost, a reduced dataset subset is used.
2. Three-Layer Neural Network (Multilayer Perceptron – MLP):
A deep learning model consisting of:
 - Input layer (Flatten)
 - Hidden Layer 1 (512 neurons, ReLU activation, dropout)
 - Hidden Layer 2 (256 neurons, ReLU activation)
 - Output Layer (10-class Softmax)

The model is optimized using the Adam optimizer and trained for 10 epochs. Both models are evaluated using accuracy metrics. KNN is expected to perform lower due to raw pixel input and high dimensionality, while the neural network performs significantly better due to its feature-learning capability.

Algorithm

A. Algorithm for K-Nearest Neighbors (KNN)

1. Load the CIFAR-10 dataset.
2. Normalize pixel values from 0–255 to 0–1.
3. Flatten each image into a 1D feature vector.
4. Select a subset of training and testing samples to reduce computational cost.

5. Initialize the KNN classifier with $k = 5$.
6. Fit the classifier using the training subset.
7. Predict labels for the testing subset.
8. Compute accuracy and classification report.

B. Algorithm for 3-Layer Neural Network

1. Load the CIFAR-10 dataset.
2. Normalize pixel values to $[0, 1]$.
3. Convert class labels to one-hot encoded vectors.
4. Build a Sequential Neural Network model with:
 - Flatten Input Layer
 - Dense Layer (512, ReLU), Dropout 0.3
 - Dense Layer (256, ReLU)
 - Dense Output Layer (10, Softmax)
5. Compile the model using:
 - Optimizer: Adam
 - Loss: Categorical Cross entropy
 - Metric: Accuracy
6. Train the network using training data for 10 epochs.
7. Validate using test data.
8. Evaluate the model and report final test accuracy.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.utils import to_categorical
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
y_train = y_train.flatten()
y_test = y_test.flatten()
```

```

print("Training samples:", x_train.shape)
print("Test samples:", x_test.shape)

x_train = x_train / 255.0
x_test = x_test / 255.0

print("\nTraining KNN classifier (this may take a minute)...")

x_train_flat = x_train.reshape((x_train.shape[0], -1))
x_test_flat = x_test.reshape((x_test.shape[0], -1))

x_train_subset = x_train_flat[:5000]
y_train_subset = y_train[:5000]
x_test_subset = x_test_flat[:1000]
y_test_subset = y_test[:1000]

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train_subset, y_train_subset)
y_pred_knn = knn.predict(x_test_subset)

print("\n--- KNN Results ---")
print("Accuracy:", accuracy_score(y_test_subset, y_pred_knn))
print(classification_report(y_test_subset, y_pred_knn))

print("\nTraining 3-Layer Neural Network...")
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

model = Sequential([
    Flatten(input_shape=(32, 32, 3)),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
history = model.fit(x_train, y_train_cat, epochs=10, batch_size=128, validation_data=(x_test,
y_test_cat))

loss, acc = model.evaluate(x_test, y_test_cat)
print("\n--- Neural Network Results ---")
print("Test Accuracy:", acc)

```

Output

```
Training samples: (50000, 32, 32, 3)
Test samples: (10000, 32, 32, 3)

Training KNN classifier (this may take a minute)...

--- KNN Results ---
Accuracy: 0.266

```

	precision	recall	f1-score	support
0	0.26	0.46	0.33	103
1	0.50	0.08	0.14	89
2	0.20	0.48	0.28	100
3	0.31	0.14	0.19	103
4	0.13	0.34	0.19	90
5	0.27	0.12	0.16	86
6	0.36	0.21	0.27	112
7	0.64	0.14	0.23	102
8	0.40	0.57	0.47	106
9	0.73	0.10	0.18	109
accuracy			0.27	1000
macro avg	0.38	0.26	0.24	1000
weighted avg	0.39	0.27	0.25	1000

```

Training 3-Layer Neural Network...

Epoch 1/10
391/391 ————— 18s 42ms/step - accuracy: 0.2275 - loss: 2.1609 - val_accuracy: 0.3402 - val_loss: 1.8048
Epoch 2/10
391/391 ————— 19s 40ms/step - accuracy: 0.3280 - loss: 1.8546 - val_accuracy: 0.3861 - val_loss: 1.7466
Epoch 3/10
391/391 ————— 15s 39ms/step - accuracy: 0.3461 - loss: 1.8003 - val_accuracy: 0.3902 - val_loss: 1.7067
Epoch 4/10
391/391 ————— 17s 42ms/step - accuracy: 0.3629 - loss: 1.7603 - val_accuracy: 0.4088 - val_loss: 1.6858
Epoch 5/10
391/391 ————— 19s 39ms/step - accuracy: 0.3791 - loss: 1.7224 - val_accuracy: 0.4218 - val_loss: 1.6330
Epoch 6/10
391/391 ————— 15s 39ms/step - accuracy: 0.3881 - loss: 1.6908 - val_accuracy: 0.4310 - val_loss: 1.6060
Epoch 7/10
391/391 ————— 20s 37ms/step - accuracy: 0.3959 - loss: 1.6698 - val_accuracy: 0.4280 - val_loss: 1.6130
Epoch 8/10
391/391 ————— 22s 40ms/step - accuracy: 0.3952 - loss: 1.6663 - val_accuracy: 0.4423 - val_loss: 1.5849
Epoch 9/10
391/391 ————— 20s 38ms/step - accuracy: 0.4001 - loss: 1.6638 - val_accuracy: 0.4332 - val_loss: 1.5968
Epoch 10/10
391/391 ————— 20s 38ms/step - accuracy: 0.4078 - loss: 1.6404 - val_accuracy: 0.4479 - val_loss: 1.5874
313/313 ————— 2s 6ms/step - accuracy: 0.4521 - loss: 1.5853

--- Neural Network Results ---
Test Accuracy: 0.4478999972343445
```

Result

Successfully implemented and evaluated two machine learning models—K-Nearest Neighbors (KNN) and a 3-layer fully connected Neural Network—on the CIFAR-10 dataset, and their classification performances were compared.

ANN USING BACKPROPAGATION ALGORITHM

Aim

To design, implement, and train a single-hidden-layer Artificial Neural Network using manual forward propagation and backpropagation, and to classify Iris flower species from the Iris dataset.

Description

Cross This experiment demonstrates the working of an Artificial Neural Network by manually coding:

- Weight initialization
- Forward propagation
- Activation functions
- Loss computation
- Backpropagation algorithm
- Weight and bias updates

The ANN is trained using the widely used Iris dataset, which contains 150 samples of flower measurements classified into three species.

A 4-6-3 neural network is built:

- Input layer: 4 neurons (sepal length, sepal width, petal length, petal width)
- Hidden layer: 6 neurons using sigmoid activation
- Output layer: 3 neurons for multi-class classification

The model is trained for 5000 epochs using Mean Squared Error (MSE) loss and evaluated using accuracy. This experiment helps understand how neural networks learn internally through gradient descent and error propagation.

Algorithm

A. Forward Propagation

1. Load and preprocess the dataset
 - Standardize input features
 - One-hot encode output labels
 - Split into training and test sets

2. Initialize weights and biases
 - Random weights for hidden and output layers
 - Bias vectors initialized to zero

3. For each epoch:
 - a. Compute hidden layer:

$$z_1 = XW_1 + b_1$$

$$a_1 = \sigma(z_1)$$

- b. Compute output layer:

$$z_2 = a_1W_2 + b_2$$

$$a_2 = \sigma(z_2)$$

- c. Compute loss using Mean Squared Error

$$\text{Loss} = \frac{1}{n} \sum (y - a_2)^2$$

B. Backpropagation

4. Compute output layer error:

$$\delta_2 = (y - a_2) \cdot \sigma'(a_2)$$

5. Compute hidden layer error:

$$\delta_1 = \delta_2 W_2^T \cdot \sigma'(a_1)$$

6. Update weights and biases using gradient descent:

$$W_2 = W_2 + a_1^T \delta_2 \cdot \eta$$

$$b_2 = b_2 + \sum \delta_2 \cdot \eta$$

$$W_1 = W_1 + X^T \delta_1 \cdot \eta$$

$$b_1 = b_1 + \sum \delta_1 \cdot \eta$$

C. Testing

7. Perform forward propagation on test data
8. Predict class with highest probability
9. Compute accuracy

Program

```
import numpy as np
```

```
import pandas as pd
```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
data = pd.read_csv(url)

X = data.drop('species', axis=1).values
y = LabelEncoder().fit_transform(data['species'])
scaler = StandardScaler()
X = scaler.fit_transform(X)
y_encoded = np.zeros((y.size, y.max() + 1))
y_encoded[np.arange(y.size), y] = 1

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2,
random_state=42)

input_neurons = X_train.shape[1]
hidden_neurons = 6
output_neurons = y_train.shape[1]

learning_rate = 0.05
epochs = 5000
np.random.seed(42)

W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.zeros((1, hidden_neurons))
W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.zeros((1, output_neurons))

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

for epoch in range(epochs):
    z1 = np.dot(X_train, W1) + b1

```

```

a1 = sigmoid(z1)
z2 = np.dot(a1, W2) + b2
a2 = sigmoid(z2)
loss = np.mean((y_train - a2) ** 2)
error_output = y_train - a2
delta_output = error_output * sigmoid_derivative(a2)
error_hidden = delta_output.dot(W2.T)
delta_hidden = error_hidden * sigmoid_derivative(a1)
W2 += a1.T.dot(delta_output) * learning_rate
b2 += np.sum(delta_output, axis=0, keepdims=True) * learning_rate
W1 += X_train.T.dot(delta_hidden) * learning_rate
b1 += np.sum(delta_hidden, axis=0, keepdims=True) * learning_rate

if epoch % 500 == 0:
    print(f"Epoch {epoch}, Loss: {loss:.4f}")

z1_test = np.dot(X_test, W1) + b1
a1_test = sigmoid(z1_test)
z2_test = np.dot(a1_test, W2) + b2
a2_test = sigmoid(z2_test)
y_pred = np.argmax(a2_test, axis=1)
y_true = np.argmax(y_test, axis=1)
accuracy = accuracy_score(y_true, y_pred)
print("\nTest Accuracy:", round(accuracy * 100, 2), "%")

```

Iris Dataset (iris.csv)

sepal.length	sepal.width	petal.length	petal.width	variety
5.1	3.5	1.4	0.2	Setosa
4.9	3	1.4	0.2	Setosa
4.7	3.2	1.3	0.2	Setosa
4.6	3.1	1.5	0.2	Setosa
5	3.6	1.4	0.2	Setosa
5.4	3.9	1.7	0.4	Setosa
4.6	3.4	1.4	0.3	Setosa
5	3.4	1.5	0.2	Setosa
4.4	2.9	1.4	0.2	Setosa
4.9	3.1	1.5	0.1	Setosa
5.4	3.7	1.5	0.2	Setosa
4.8	3.4	1.6	0.2	Setosa
4.8	3	1.4	0.1	Setosa
4.3	3	1.1	0.1	Setosa
5.8	4	1.2	0.2	Setosa
5.7	4.4	1.5	0.4	Setosa

Output

```
Epoch 0, Loss: 0.2413
Epoch 500, Loss: 0.0114
Epoch 1000, Loss: 0.0103
Epoch 1500, Loss: 0.0099
Epoch 2000, Loss: 0.0096
Epoch 2500, Loss: 0.0091
Epoch 3000, Loss: 0.0085
Epoch 3500, Loss: 0.0081
Epoch 4000, Loss: 0.0076
Epoch 4500, Loss: 0.0073
```

```
Test Accuracy: 100.0 %
```

Result

Successfully implemented an Artificial Neural Network (ANN) using manual Backpropagation on the Iris dataset and achieved accurate classification performance. The model was trained using forward propagation, Mean Squared Error loss, and gradient-based weight updates, and the final accuracy was computed on the test dataset.

LOGISTIC REGRESSION

Aim

To implement a Logistic Regression model on Diabetes Dataset and to evaluate the model using accuracy, precision, recall, and F1-score.

Description

Logistic Regression is a supervised classification algorithm used to model the probability of a binary outcome. In this experiment, Logistic Regression is applied to the Diabetes Dataset to predict whether a patient has diabetes based on several medical predictor variables. The dataset is loaded and preprocessed into input features (X) and target labels (y). The data is then divided into training and testing subsets using an 70–30 split. The Logistic Regression model is trained using the training set and evaluated on the test set using standard classification metrics such as accuracy, precision, recall, and F1-score.

Algorithm

- 1: Import necessary libraries (Pandas, NumPy, Scikit-learn).
- 2: Load the diabetes dataset (CSV file) into a Pandas DataFrame.
- 3: Separate input features (X) and target label (y = Outcome).
- 4: Split the dataset into training and testing sets using `train_test_split()`.
- 5: Initialize the Logistic Regression model with suitable parameters (`maxiter = 1000`).
- 6: Train the model using the training data (`model.fit()`).
- 7: Predict output for test data using `model.predict()`.
- 8: Evaluate the model using: Precision Recall, F1-score and Accuracy
- 9: Display the performance metrics

Program

```
import pandas as pd
import numpy as np

df = pd.read_csv("diabetes.csv")
df.head()
x = df.drop (columns = 'Outcome')
y = df['Outcome']
from sklearn.model_selection import train_test_split
x_train, x_test,y_train, y_test = train_test_split(x,y, test_size = 0.30, random_state = 42)
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(x_train, y_train)
y_pred = model.predict (x_test)

from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
```

```

pr = precision_score(y_test, y_pred)
re = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
ac = accuracy_score(y_test, y_pred)
print('precision :', pr)
print('recall_score :', re)
print('f1_score :', f1)
print('accuracy', ac)

```

Diabetes Dataset (diabetes.csv)

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesProbability	Age	Outcome
6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1
5	116	74	0	0	25.6	0.201	30	0
3	78	50	32	88	31	0.248	26	1
10	115	0	0	0	35.3	0.134	29	0
2	197	70	45	543	30.5	0.158	53	1
8	125	96	0	0	0	0.232	54	1
4	110	92	0	0	37.6	0.191	30	0
10	168	74	0	0	38	0.537	34	1
10	139	80	0	0	27.1	1.441	57	0
1	189	60	23	846	30.1	0.398	59	1
5	166	72	19	175	25.8	0.587	51	1

Output

```

precision : 0.6172839506172839
recall_score : 0.625
f1_score : 0.6211180124223602
accuracy 0.7359307359307359

```

Result

Successfully implemented Logistic Regression on the Diabetes Dataset and the evaluation metrics-accuracy, precision, recall, and F1-score were computed to assess the classification performance.

FAMILIARISATION OF GOOGLE COLAB

Aim

To familiarise cloud computing like google colab

Overview

- Understand what is Google Colab
- Get a list of top Alternatives to Google Colab
- By no means is this list exhaustive. Feel free to add more in the comments below.

Introduction

For anyone who has storage issues or is not in a position to afford a system compatible to their requirements for data science work, Google Colab has been a blessing. Working with Colab has opened up so many avenues for me that I thought weren't possible before. We no longer have the restriction of poor computational power on our machines. And Free GPUs are at our fingertips. But as a data scientist it is important to know all the alternatives available for a particular tool. So, in this article, we will be exploring some of the widely used alternatives to google colab.

What is Google Colaboratory?

Google colaboratory or Google Colab is a free Jupyter Notebook environment. It is a free cloudbased service by Google which means you don't have to pay anything. One of the best things about Colab is that you don't need to install anything beforehand. In fact, many of the Data Science and Machine Learning libraries such as Pandas, NumPy, Tensorflow, Keras, OpenCV come pre-installed with Colab. The notebooks you create are saved on your Google Drive. So Colab also leverages the collaboration features of Google Docs, where you can share your notebook with multiple people easily and all of you can work on the same notebook at the same time without any issue. Google also provides the use of a free NVIDIA Tesla K80 GPU. If you connect Colab to Google Drive, that will give you up to 15 GB of disk space for storing your datasets. You can run the session in an interactive Colab Notebook for 12 hours, which is enough for a beginner. Google has its self-made custom chips called TPUs. One more thing to keep in mind is that the dataset you upload in the Colab notebook gets deleted once the session is ended. However, you can upgrade to the Pro version, which supposedly gives you access to faster GPUs – NVIDIA TESLA T4 or P100 GPUs, longer runtimes up to 24 hours, and more RAM.

Alternatives of Google Colab

In the following section, we discuss the top 4 alternatives to google colab

1. Amazon SageMaker

Amazon sagemaker is also a cloud-based Machine Learning platform developed by Amazon in November 2017. It provides hosted Jupyter notebooks that require no setup. But it is not free. Yes, you have to pay for its services, though the trial is free (for the initial two months).

“Using SageMaker Studio is free, you only pay for the AWS services that you use within Studio.”

Pros:

- Along with the Deep Learning frameworks like Tensorflow, scikit learn, PyTorch, and XGBoost, which are provided by Google Colab, SageMaker provides MXNet, Chainer, and SparkML too.
- It offers the following features- Amazon SageMaker Ground Truth, Amazon Augmented AI, Amazon SageMaker Studio Notebooks, Preprocessing, Amazon SageMaker Experiments, and many more.

Cons:

- If you train your model using built-in algos of SageMaker, you cannot deploy it outside SageMaker. This is also the case for Google’s AutoML, though all models trained on MLengine (including those using Google’s TensorFlow-hub modules) can be deployed anywhere.
- The automatic hyperparameter optimization works better in Colab, in terms of results produced and time taken.
- You get new versions of Tensorflow on SageMaker weeks after you get them on Colab.

2. CoCalc

CoCalc or Collaborative Calculation is a web-based cloud computing (SaaS) and course management platform for computational mathematics. It is an open-source software hosted by SageMath Inc. The creator and lead developer of CoCalc are William Stein, a former professor of mathematics at the University of Washington. Along with the Jupyter notebook it supports editing of Sage worksheets and Latex documents.

Pros:

- It offers real-time collaboration, which means you can share your notebook with others and you all can edit it at the same time.
- For the free plan offered by CoCalc- Sessions will shut down after 30 minutes of inactivity, though they can run for up to 24 hours, which is twice the time offered by Colab.
- It has a history recording feature that records all of your changes to the notebook in fine detail and allows you to browse those changes using an intuitive slider control.
- Languages offered- Python, Sage, R, Octave, and many more.

Cons:

- The service is not free. However, there is a free plan but a Trial Project with certain restrictions – most notably your project runs with lower hosting quality and has no access to the internet to download data from other servers.
- GPU is not available, neither in the free plan nor in the upgraded version.

3. Kaggle Kernel

Kaggle is a popular platform for its Data Science Competitions, however, they also provide free Kernels or Notebooks for performing all the Machine Learning and Data Science tasks, independent of the competitions. Kaggle Kernels is a free platform to run Jupyter notebooks in the browser. Both Colab and Kaggle are the product of Google and have many similarities.

Kaggle has updated its kernels to have more computation power and memory. 20GB Dataset, 5GB Disk Space, 9 hours run time, and 4 CPUs w/ 16GB RAM or when the GPU is turned on it is 2 CPU cores w/ 13GB RAM.

Pros:

- Kaggle provides free access to NVIDIA TESLA P100 GPUs in kernels. This benchmark shows that enabling a GPU to your Kernel results in a 12.5X speedup during the training of a deep learning model.
- It supports two of the main languages in the field of Data Science- R and Python.
- Most keyboard shortcuts from Jupyter Notebook are almost similar in Kaggle Kernels, which makes it easier for a person working in Jupyter Notebooks to work in Kaggle.
- Kaggle has a large community to support, learn, and validate data science skills.

Cons:

- In general, Kaggle has a lag while running and is slower than Colab.
- Kaggle typically limits kernel running time to 9 hours, with time out after 1 hour of inactivity.
- A major drawback of both platforms is that the notebooks cannot be downloaded into other useful formats.

4. Binder

Binder is powered by Binderhub, which is an open-source tool that deploys the Binder service in the cloud. Binder allows you to create custom computing environments that can be shared and used by many remote users. It allows you to input the URL of any public Git repository, and it will open that repository within the native Jupyter Notebook interface. You can run any notebooks in the repository, though any changes you make will not be saved back to the repository.

It can be helpful when you have a repository full-on Jupyter Notebooks. Although there is a user limit of 100 users for a repository (which is enough I guess).

Pros:

- Languages supported- Python, R, and Julia.
- Since it is an open-source project, it is free.
- Binder can run your notebooks directly from GitHub.

Cons:

- Collaboration with others is not available.
- Sessions will shut down after 20 minutes of inactivity, though they can run for 12 hours or longer.
- Not suitable while working with large datasets.

Result

Familiarise cloud based computing using google colab and other alternatives.